Chapter 4

Addressing Modes (part 2)

Pre-index and Post-index

- In **pre-index**, the indirect register is **autoindex** **before** being used to compute effective address.

```
LDR R1,[R0,#4]!   ; R0 = R0+4
                  ; R1 = mem[R0]
```

Offset with Autoindexing (pre-index)

```
LDR R1,[R0,R2]!   ; R0 = R0+R2
                  ; R1 = mem[R0]
```

Index with Autoindexing (pre-index)

- In **post-index**, the indirect register is used to compute the effective address **before** it is **autoindexed**.

```
LDR R1,[R0],#4    ; R1 = mem[R0]
                  ; R0 = R0+4
```

Offset with Autoindexing (post-index)

```
LDR R1,[R0],R2    ; R1 = mem[R0]
                  ; R0 = R0+R2
```

Index with Autoindexing (post-index)

Auto Indexing

Given the shown initial states, what is the value of R0 and R1 after executing LDR R1, [R0,#4]!

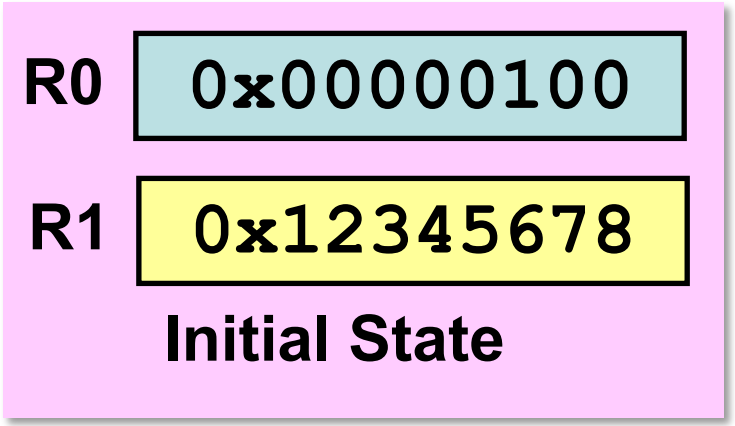
A) R0= 0x00000100 R1=0x12345678

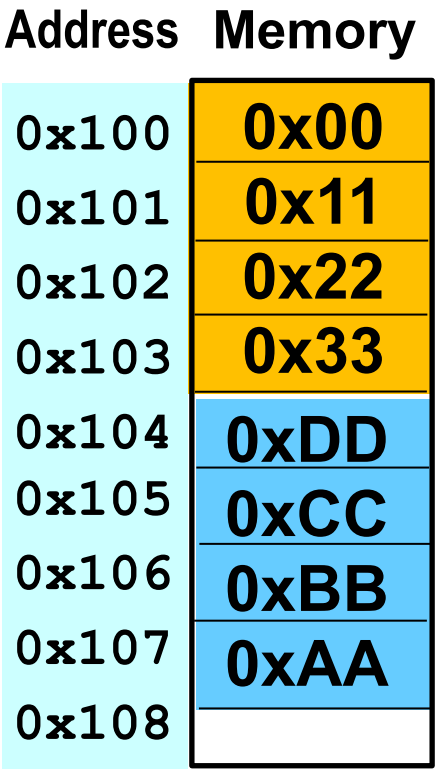
B) R0= 0x00000104 R1=0x33221100

C) R0= 0x00000104 R1=0xAABBCCDD

D) R0= 0x00000100 R1=0xAABBCCDD

Answer is C





Auto Indexing

Which registers are affected by executing LDR R1, [R0],#4

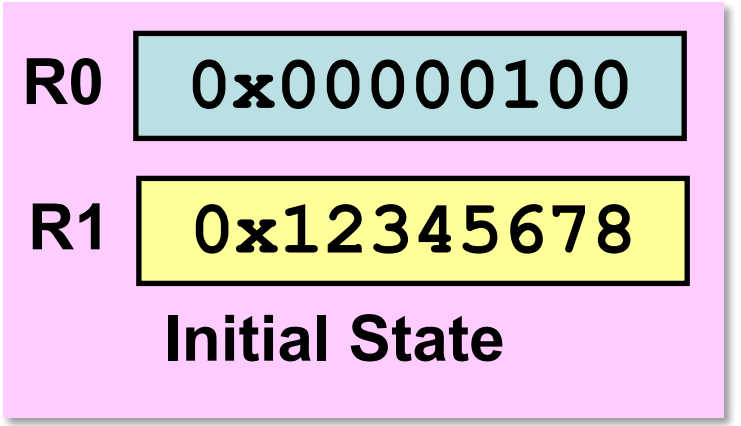
A) R1

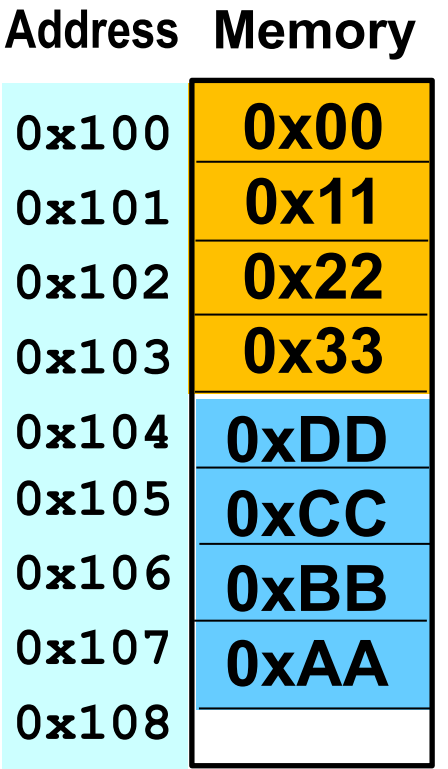
B) R1,PC

C) R1,R0,PC

D) R1,R0,PC,CPSR

Answer is C





ARM Stack Implementation

Based on the description of the Push and Pop operations shown in pre-recorded lectures, which ARM stack type is being implemented here

```
Push- STR R1,[SP,#-4]!
Pop - LDR R1,[SP],#4
```

A) Full Descending (FD)

B) Empty Descending (ED)

C) Full Ascending (FA)

D) Empty Ascending (EA)

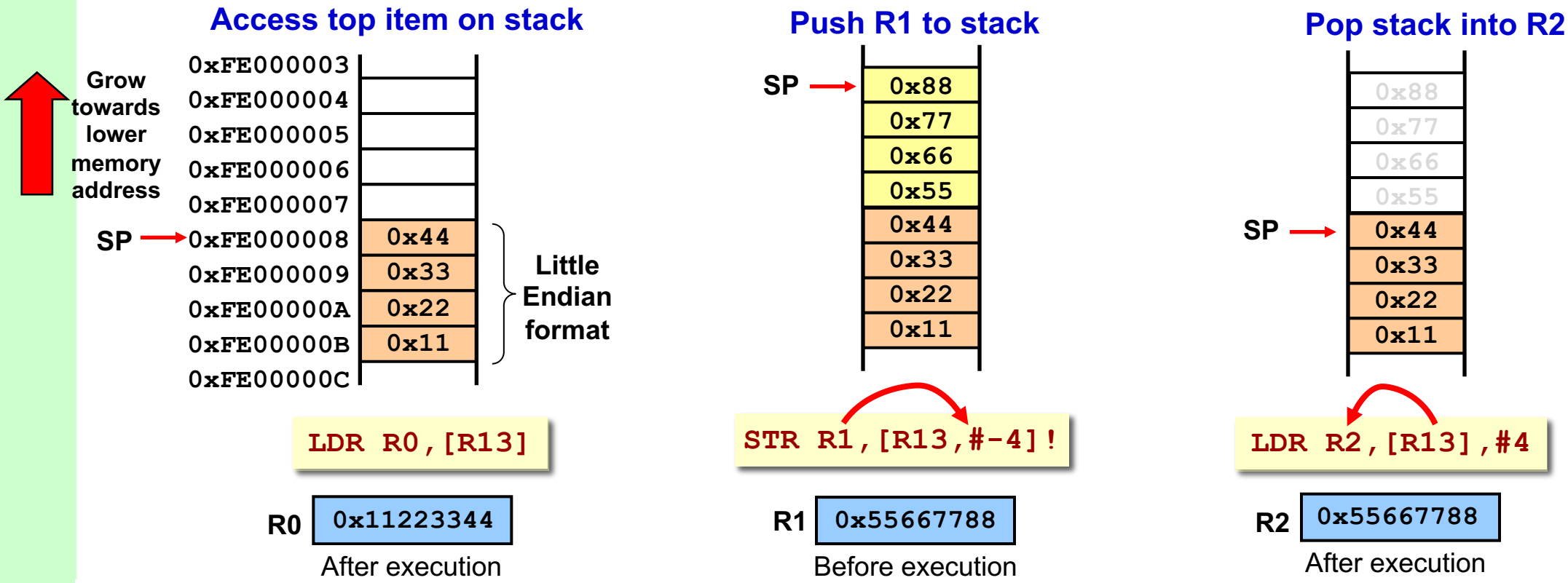
Answer is A

ARM Stack Implementation

Based on the description of the Push and Pop operations shown in pre-recorded lectures, which ARM stack type is being implemented here

```
Push- STR R1,[SP,#-4]!
Pop - LDR R1,[SP],#4
```

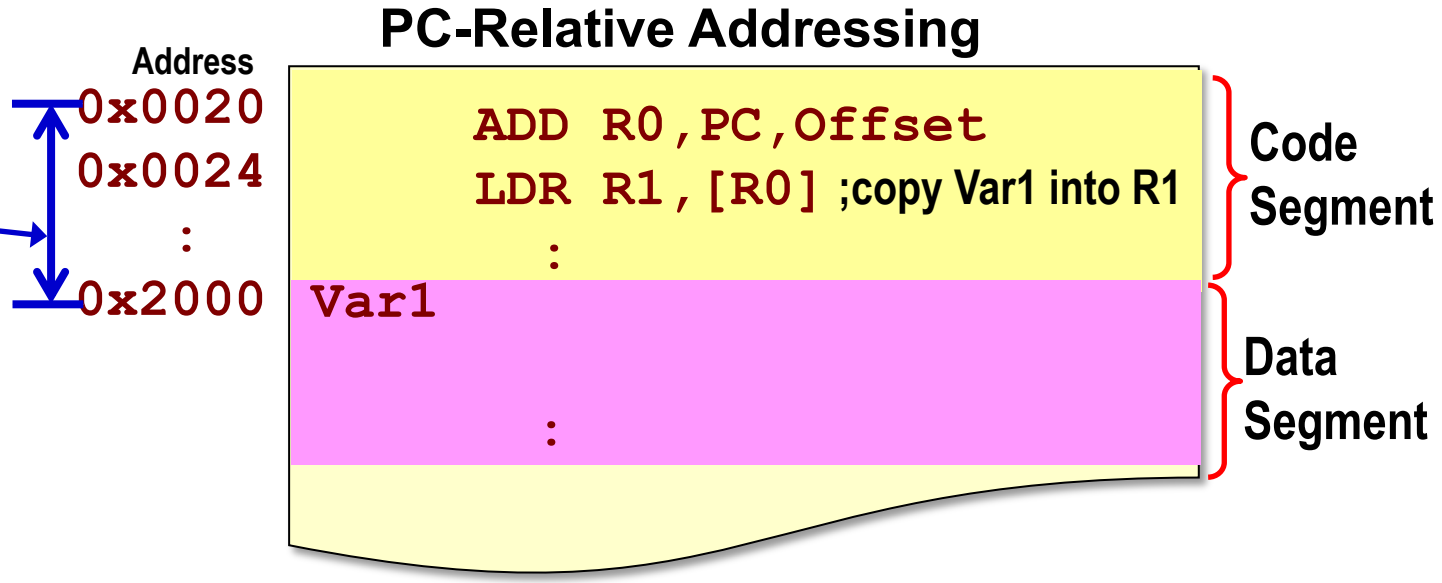
Example of **Full Descending** (`FD`) stack implementation:



PC-relative Data Access

PC-relative offset is added since PC has incremented by 8 when executing ADD instruction.

PC-relative Offset:
Var address – (PC value + 8)



Note: In the ARM processor the **PC** points **8** bytes ahead of the current executed instruction.

A) 0x2000

B) 0x2000 – 0x0020

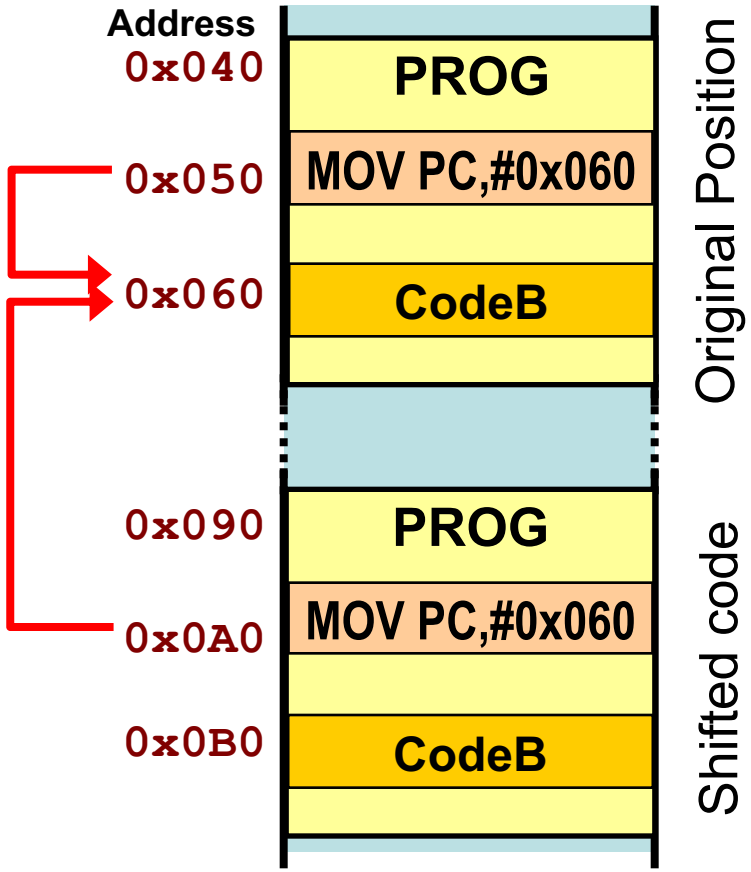
C) 0x2000 – 0x0020 + 8

D) 0x2000 – (0x0020 + 8)

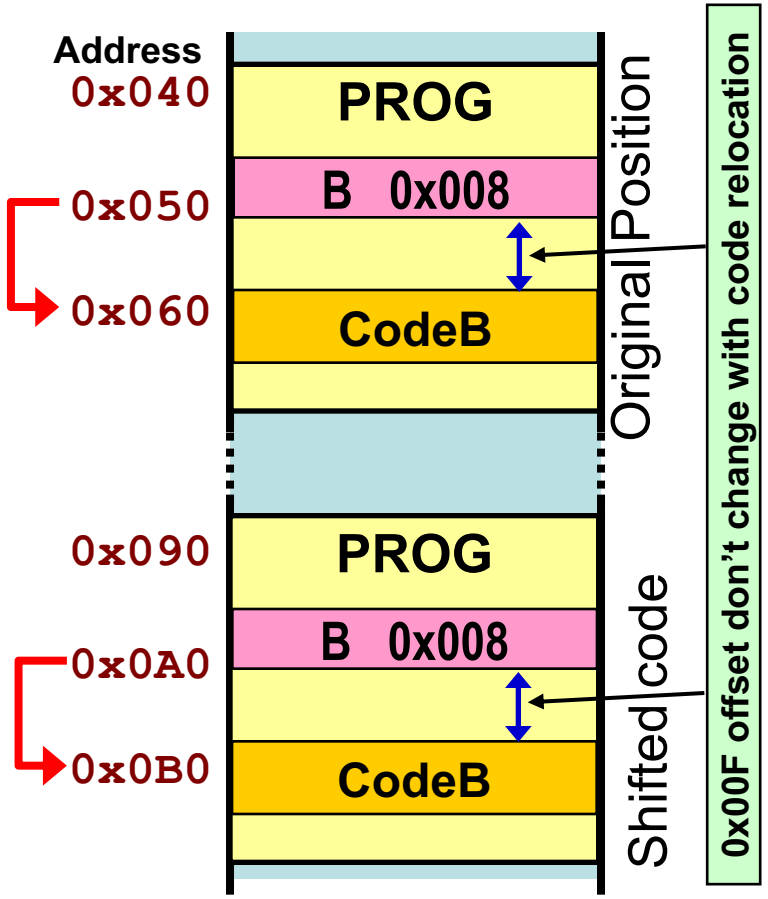
Answer is D

Position-Independent Code

- Such programs can be loaded anywhere in memory and still execute correctly (i.e. relocatable).



Does not jump to **CodeB** after **PROG** is shifted to `0x090`



`B` still branch to **CodeB** after **PROG** is shifted to `0x090`